

PersonaPage

Complete Reference Document

Product Strategy · Architecture · Engineering Plan · PRD

London Tech Week MVP · Solo Builder Edition

"A world where every link you share shows exactly the right version of you to exactly the right person."

Product Decisions

Core mechanic	Context-first smart links from one master profile
Wedge user	CS students & early-career developers
Primary outcome	Stronger personal brand → more opportunities
Stack	Next.js 15 · TypeScript · Supabase · OpenAI · Vercel
Target	Demo-ready MVP for London Tech Week
Build time	~2 weeks, solo developer

Contents

01 Product Vision & Problem	09 Database Schema
02 Core Thesis & ICP	10 API Design
03 User Personas	11 Frontend Architecture
04 Positioning & Competitive	12 Backend Architecture
05 Product Direction	13 Build Order
06 User Flows	14 Technical Risks
07 Feature Scope	15 MVP Scope & Roadmap
08 AI System Design	16 Validation Metrics

Vision

"A world where every link you share shows exactly the right version of you to exactly the right person."

Professionals lose opportunities not because they lack skill — but because they present the same static identity to every audience. PersonaPage makes professional identity as contextual as a real conversation.

The contrarian bet

"The problem is not that people cannot write about themselves. The problem is that one link was never supposed to work for everyone. Professional identity should be audience-aware by default."

Problem Statement

The Situation	CS students and technical builders share the same portfolio/CV universally — to startup founders, FAANG recruiters, design leads, and collaborators alike.
The Trigger	The moment you paste your GitHub link into a cold message and think: "This doesn't quite fit what they're looking for."
Current Behaviour	(a) Maintain 3–4 manually updated versions. (b) Send the generic version hoping it lands. (c) Spend 30+ minutes rewriting per outreach.
Why Tools Fail	LinkedIn: one profile, no adaptation. Notion: static, manual. Resume builders: file output, not a live link. None treat the viewer as a variable.
The Cost	Tangible: missed interviews, poor cold-outreach response rates. Intangible: the anxiety of knowing your work is better than your presentation shows.
Frequency	Every job application. Every cold DM. Every hackathon intro. For an active networker: 5–15 times per week.

The Contrarian Belief

Every other portfolio tool optimises for how YOU present yourself. PersonaPage optimises for how THEY perceive you. The viewer is the variable — not the content.

The Core Mechanic

User creates ONE master profile. AI generates N smart links, each pre-configured for a different audience. Each link renders a different framing from the same underlying data.

The Compounding Mechanic

More projects → richer AI source material. More links shared → analytics on what resonates. The product becomes more valuable the more it is used.

90-Day Falsifiability Test

If users never share a second contextual link beyond their first, the thesis is wrong. It means they need one better version, not multiple. That outcome pivots the product toward a smarter single-profile AI builder.

Ideal Customer Profile

Primary Segment: CS students in their final 1–2 years, or recent grads (0–2 years), actively job hunting, freelancing, or building side projects — who care about being taken seriously as builders.

Age range	20–26
Career stage	Student → junior / early career
Technical level	Comfortable with tools; wants zero-config UX
Context	University, hackathons, tech Twitter/X, Discord builder communities
Cares about	Being perceived as a real builder, not just a student
Afraid of	Being overlooked because their presentation is weak
Will switch when	Gets ghosted after sending a generic GitHub link that told the wrong story
Won't adopt if	AI output doesn't sound like them, or setup takes more than 10 minutes
Where to find	Uni CS Discord/Slack, MLH/Devpost hackathons, r/cscareerquestions, #buildinpublic

Aryan — The Overlooked Builder

Final-year CS student, applying to startups

"His GitHub has 40 repos. His CV makes him look like every other student."

Primary goal	Land a SWE internship or junior role at a startup
Frustration	Startup recruiters want product impact; FAANG wants structured CV — same link fails both
Trigger	Gets rejected after a strong technical round. Realises his portfolio led with coursework, not his best project.
Success	Gets a reply from cold outreach within 48 hours — his AI project link sent to the right recruiter actually landed.
Churn risk	AI-generated versions sound generic and don't reflect his voice

Priya — The Multi-Hat Founder

Recent CS grad, freelancing + building a startup + seeking co-founder

"She's three different people professionally — and has one LinkedIn that captures none of them."

Primary goal	Find a technical co-founder who matches her vision
Frustration	Founder networking needs a completely different story than client pitching — same link fails both
Trigger	Heading to a tech meetup needing to share her profile with a potential client AND co-founder in the same evening.
Success	Creates two smart links before the event, one per audience, and follows up knowing exactly what each person saw.
Churn risk	Managing multiple versions becomes more work than writing manually

Recommended Positioning

Hero headline: Stop sending the same portfolio to everyone. PersonaPage gives each person the version of you that actually speaks to them.

Tagline: Smart links for your professional identity.

Competitive Landscape

Tool	Does well	Structural miss
LinkedIn	Distribution & credibility	One profile, zero audience adaptation
Read.cv	Clean aesthetic, builder community	Static, no AI layer
Notion portfolio	Flexible layout, free tier	No audience awareness, fully manual
Polywork	Multi-role identity framing	Manual, no smart links
Resume builders	CV formatting & templates	File output, not a live link
Linktree	Simple link aggregation	Zero identity depth

The unowned gap: Nobody owns: live link + adaptive profile + audience context + per-link analytics. Linktree owns links. LinkedIn owns identity. Nobody owns the intersection.

Honest threat: LinkedIn could ship 'audience modes' tomorrow. The moat in Year 1 is UX focus and speed of iteration for this specific user pain — not technology.

Core Decision: Link-as-Product

The profile is the backend. The link is the product. Users do not share their profile. They share links. Analytics attach to links. Virality happens when a link is opened by someone who has never heard of PersonaPage.

Onboarding Flow — target: first link live in under 8 minutes**0 Sign up**

1 Email or GitHub OAuth → username selection

0 Build master profile

2 Headline + 1–5 projects minimum (4 sections, all optional except headline)

0 Pick audience context

3 Three cards: Recruiter / Founder / Freelance Client

0 AI generates view

4 Live preview rendered in 3–5 seconds

0 First smart link ready

5 yourname.personapage.io/p/username — copy button, prominent

0 Add second context

6 Most users will — the first took under 2 minutes

Core Usage Loop

- User wants to message someone → opens dashboard (5 seconds)
- Copies the correct context link (recruiter / founder / client)
- Pastes into LinkedIn DM / email / Discord / job application
- Done — total time: 10–15 seconds
- Later: email notification 'Your /recruiters link was opened 3 times'
- User opens dashboard to check analytics → updates a project
- ALL contextual links automatically reflect the update

TIER 1 — Build first (Week 1–4)

- ✓ Master profile builder (headline, projects, skills, experience, tone)
- ✓ Audience context templates: recruiter, founder, client, general
- ✓ AI generation of contextual view per link
- ✓ Unique shareable link per context (personapage.io/p/username)
- ✓ Basic analytics: view count, location, device, referrer
- ✓ View notification email (Resend — triggered on new view)

TIER 2 — After first 10 users (Week 5–8)

- ✓ Custom context creator (user-defined audience type)
- ✓ GitHub project import (auto-populate projects from repos)
- ✓ Visual profile customisation (colours, layout variants)
- ✓ Link expiry and access control
- ✓ Analytics: session depth, referrer breakdown

NEVER BUILD (MVP)

- ✗ Mobile app (responsive web is sufficient)
- ✗ Team or org accounts
- ✗ Resume PDF export (that is not this product)
- ✗ Social feed or profile discovery
- ✗ Real-time audience auto-detection
- ✗ Payments (build when 50 users ask how to pay)
- ✗ Blockchain / decentralised identity

Non-Goals

Not a resume formatter	PersonaPage is a live link, not a file output
Not a job board	No recruiter-facing discovery or job listings
Not a social network	No feed, following, or public profile directory
Not a LinkedIn replacement	Complementary tool, not a competitor

Not for enterprises	Solo professionals only for MVP
No auto-viewer detection	Viewer intent detection is a research problem — let users pre-configure contexts

What stays fixed (AI never modifies)

Fixed across all contexts: user's name · project titles · technology names · GitHub/LinkedIn URLs · any metric the user provided ('40% faster', '2,000 users') · dates, institutions, company names. Metrics are facts — the AI is explicitly instructed never to invent them.

What changes per context

Headline framing	Same person, different narrative angle per audience
Project ordering	Most relevant to audience shown first
Project description	Same project, different emphasis — technical vs. product vs. delivery
Skills ordering	Most relevant skills surfaced first
CTA text	Different ask per audience type
Section order	About / projects / skills priority varies by context

Context Configuration

Context	Audience	Focus	Tone
recruiter	Hiring Manager	Technical credibility, shipped products, measurable outcomes	Professional and direct
founder	Startup Founder	Product thinking, builder mentality, ownership, initiative	Peer-to-peer, builder to builder
client	Freelance Client	Delivery track record, reliability, scope management	Trustworthy and professional
general	General audience	Balanced overview, approachable	Clear and approachable

Hallucination Prevention — three layers

Layer 1: Schema enforcement	AI always outputs JSON. Validated against Zod schema on receipt. Any field not in source data is omitted.
Layer 2: Fact anchoring	Every prompt explicitly lists ground-truth fields: 'These facts must appear exactly as provided or be omitted entirely — never modify, round, or extrapolate.'
Layer 3: Title validation	Code removes any project the AI invented by cross-checking generated titles against source project titles.

Caching Strategy

generated_content table IS the cache. One row per link. Updated on explicit regeneration. Read on every public page load. Do NOT call OpenAI on page view. At demo scale, Supabase reads are fast enough — no Redis, no CDN cache on AI output needed.

Retry Logic

Max attempts	3 attempts before returning error to client
Backoff	1s, 2s, 3s exponential between attempts
API timeout	30 seconds (Vercel function default is sufficient)
Parse failure	Retry with same prompt — JSON failures are transient
Schema failure	Retry — Zod validation failure means malformed output
Partial output	Never save — only upsert on full schema validation pass

Tables Overview

Table	Purpose	Key constraint
users	App-level user data, mirrors auth.users	username UNIQUE, references auth.users
profiles	Master profile — single source of truth	UNIQUE user_id (one per user)
projects	Ordered array of projects per profile	display_order for manual sorting
skills	Flat skill tags with category	display_order, category enum
links	Each shareable contextual link	UNIQUE (user_id, slug)
generated_content	Cached AI output per link	UNIQUE link_id — upserted on regen
page_views	Analytics — append-only, never updated	is_owner flag to filter self-views

Entity Relationships

auth.users	→ 1:1 →	public.users
public.users	→ 1:1 →	profiles
profiles	→ 1:N →	projects
profiles	→ 1:N →	skills
public.users	→ 1:N →	links
links	→ 1:1 →	generated_content
links	→ 1:N →	page_views

Critical Indexes

idx_users_username	Public page resolution — most critical path
idx_links_user_slug	Link lookup by (user_id, slug)
idx_page_views_link_time	Analytics queries: (link_id, viewed_at DESC)
idx_projects_profile_order	Profile load: (profile_id, display_order)

RLS Summary

users	Own record only (auth.uid() = id)
profiles	Own for write, public SELECT allowed
projects / skills	Own for write via profile_id → user_id chain; public SELECT
links	Own for write; public SELECT on is_active = true
generated_content	Own for write; public SELECT

page_views

Service role key for INSERT (viewer is not authed); own SELECT

Endpoints

Method + Route	Auth	Purpose
POST /api/ai/generate	Required	Trigger AI generation for a link
POST /api/analytics/view	None (service role)	Record a public page view
GET /api/analytics/[linkId]	Required	Fetch analytics for dashboard
POST /api/links	Required	Create a new context link

POST /api/ai/generate

Request body	{ link_id: string }
Response 200	{ content: GeneratedContent }
Response 400	{ error: 'MISSING_LINK_ID' 'NOT_OWNER' 'PROFILE_INCOMPLETE' }
Response 422	{ error: 'GENERATION_FAILED' 'PARSE_FAILED' 'SCHEMA_INVALID' }
Response 503	{ error: 'AI_UNAVAILABLE' }
Side effect	Upserts generated_content table on success

POST /api/analytics/view

Request body	{ link_id, referrer, device: 'mobile' 'desktop' 'tablet' }
Response	Always 200 — { ok: true } — never expose errors
Server adds	country + city from Vercel headers (x-vercel-ip-country, x-vercel-ip-city)
Auth model	Uses SUPABASE_SERVICE_ROLE_KEY server-side — never in client code
Side effect	Triggers view notification email via Resend (async, non-blocking)

GET /api/analytics/[linkId]

Response 200	{ total_views, views_last_7_days, views_by_day[], recent_views[], top_referrers[] }
views_by_day	Last 30 days, one entry per day: { date, count }
recent_views	Last 20 views: { viewed_at, country, city, device, duration_seconds }
top_referrers	Top 5 referrer domains with count

Error Handling Convention

String error codes everywhere. No stack traces to client. No raw error messages. Log server-side, return a code string. ok(data) for 200, err('CODE', status) for errors. This is the entire error handling system for V1.

Folder Structure Philosophy

lib/ is flat. components/ is grouped by feature. types/ is one file. When you open Cursor and say 'edit the AI generation logic,' it finds lib/ai.ts immediately. There is no archaeology. Feature-grouped components mean Cursor's context works with adjacent, related files.

Route Map

Route	Type	Purpose
/	Public	Landing + redirect to dashboard
/login /signup	Public	Auth pages
/p/[username]	Public SSR	Shareable profile page
/dashboard	Protected	Link list + overview
/profile	Protected	Master profile editor
/links	Protected	Manage context links
/links/[id]	Protected	Per-link analytics
/settings	Protected	Account settings (stub)

SSR vs CSR Decision Rules

Server Components	Everything that only reads data and renders it. All layouts. All page root components. Public profile page. Dashboard page. Analytics page.
Client Components	Add 'use client' ONLY for: form inputs (onChange), clipboard API, recharts (needs browser), toast notifications, loading state during API calls, inline edit toggle.
The rule	If a component has no onClick / onChange / useState / useEffect — it is a Server Component. Never make a layout a Client Component.
State management	No Zustand. No Redux. No React Query for V1. Server Components + useState + React Hook Form covers everything.

URL Decision: /p/[username] not /[username]

Using /p/[username] avoids route collision. If user profiles were at root, then /dashboard, /login, /api become reserved words. Next.js catches [username] as a dynamic segment before static routes in some configurations, causing confusing 404 behaviour. /p/ is clean, clear, and safe.

Component Organisation

components/ui/	Primitives only: Button, Input, Card, Badge, Spinner, EmptyState
components/dashboard/	Sidebar, TopBar, LinkCard, StatsBar

components/profile/	ProfileForm, ProjectItem, SkillsInput, ToneSelector
components/public/	PublicProfile, ProjectCard, SkillsDisplay, ViewCapture
components/analytics/	ViewsChart (CSS-based), RecentViews
lib/	ai.ts, prompts.ts, supabase.ts, analytics.ts, validations.ts, utils.ts
hooks/	useProfile.ts, useCopyLink.ts, useAnalytics.ts

Day 1

Foundation

- create-next-app with TypeScript + Tailwind + ESLint + App Router
- Install: supabase-js, auth-helpers-nextjs, openai, zod, react-hook-form, sonner, resend
- Create Supabase project, run migrations 001–003
- Set env vars locally and on Vercel, deploy empty project
- Test Supabase connection from a route — confirm everything works

Days 2–3

Auth + User Setup

- Login and signup pages with email + password
- GitHub OAuth (30 minutes, worth it for demo)
- On signup: create public.users + profiles rows
- Dashboard stub showing 'you are logged in as X'
- Sidebar layout shell

Days 4–5

Profile Editor

- Profile page: fetch profile + projects + skills server-side
- ProfileForm with all sections + Server Action save
- ProjectItem: add, edit, remove, reorder (arrows — not drag)
- SkillsInput: comma-separated tag input
- ToneSelector: three-option radio

Days 6–7

AI Generation

- lib/ai.ts: full generation function with Zod validation + retry
- POST /api/ai/generate: auth + ownership check + upsert
- Links page: list links, create new link form
- Generate button per link with spinner and result preview
- CRITICAL: fix prompt quality NOW before proceeding to public page

Days 8–9

Public Page

- `/p/[username]` full SSR — resolve username → link → content
- `PublicProfile` component: all sections rendered cleanly
- `ViewCapture`: client component, fires once on mount
- `POST /api/analytics/view`: records view with geo data
- 'Made with `PersonaPage`' footer for growth loop

Days 10–11

Dashboard + Analytics

- Dashboard: link list with view counts + copy button
- `/links/[id]` analytics page: chart + recent views
- CSS bar chart (no library needed in V1)
- View notification email via Resend

Days 12–13

Polish

- Empty states: no links, no views, no profile data
- Error boundaries and fallback UI for generation failure
- Mobile responsive layout throughout
- Page metadata + OpenGraph for public pages
- Rate limiting on `/api/ai/generate` (10/day per user)

Day 14

Demo Prep

- Seed 3 realistic demo profiles
- Time the full flow: signup → profile → generate → share link
- Target: under 5 minutes from signup to shareable link
- Fix the top 3 friction points — do NOT add features

Critical Risks (in order of danger)

AI output sounds generic	SEVERITY: HIGH. Will kill trust immediately. Test prompt with 5 real profiles before building UI. If output sounds like a ChatGPT CV, fix the prompt — do not proceed.
Cool demo syndrome	SEVERITY: HIGH. Users generate once and never return. Fix: the link is the product, not the profile. View notification email is the return mechanic.
Service role key in client code	SEVERITY: CRITICAL. Immediate security vulnerability. SUPABASE_SERVICE_ROLE_KEY must never appear in any NEXT_PUBLIC_ variable or client component.
No content on first public view	SEVERITY: MEDIUM. Generated content row may not exist when the link is first shared. Always handle null case with graceful 'Profile being set up' state.
Slug collision not handled in UI	SEVERITY: MEDIUM. DB constraint prevents it but throws a 500. Add try/catch + 'slug already taken' user-facing error message.

Common Next.js Mistakes to Avoid

useEffect for data fetching	WRONG. Fetch in Server Components, pass as props. Never useEffect() => fetch('/api/data'), [] in a page component.
'use client' on layouts	Making a layout client just because one child needs interactivity. Extract the interactive part into its own Client Component file.
AI call on public page load	Every visitor triggers an OpenAI call. generated_content IS the cache. Public page reads from it — never calls AI.
Loading skeletons before features work	Skeletons are polish. Build the feature, confirm it works, then add loading states. Many beginners spend day 6 on skeleton animations for a dashboard with no real data.

Technical Debt: Acceptable vs. Dangerous

Acceptable in V1	Dangerous — fix before launch
String error codes not typed classes	Service role key in any client code
Inline Supabase queries in page files	No handling for missing generated_content
console.error for logging	No username validation in signup form
CSS bar chart instead of recharts	Calling OpenAI on unvalidated profiles
Plain text emails no HTML template	Analytics insert using anon key (no RLS)
Single types/index.ts file	No rate limiting on generation endpoint

Things That Should NOT Be Abstracted Early

Do not abstract: The prompt builder (keep as a function, not a class). Supabase queries (write them inline in Server Components). Error types (string codes are fine). The generation pipeline (one function, not a service). Email sending (one Resend call in the analytics route). The abstraction cost is paid upfront. The benefit only exists when the abstraction needs to change. In V1, almost nothing needs to change — you need it to work.

The single most important metric

Average number of unique context links shared per active user per month. If above 3: core thesis confirmed — double down. If equals 1: thesis may be wrong — users want one better version. If zero after two weeks: onboarding is broken — fix before anything else.

Week-by-week validation targets

Week	Metric	Target
W1–2	Onboarding completion rate	> 60% reach 'link created'
W2–4	Links created per user	> 2 within first session
W3–5	Links shared (copied) per user	> 1 within first week
W4–6	Return visits within 7 days	> 40%
W5–8	View notification email open rate	> 50%
W6–8	Profile update rate	> 30% update at least once
W8+	Links shared per active user per month	> 3 — thesis confirmed
W10+	'Made with PersonaPage' CTR	> 5% of link viewers

The Pivot Trigger

If by end of Month 3: average links shared per user is consistently 1 or below, users do not return after initial generation, and qualitative feedback shows 'I just copied the text and used it elsewhere' — strip the adaptive features and rebuild as the sharpest AI-powered single portfolio tool for CS students. Different product, same user, same underlying AI capability.

Final Architecture Principles

1. Build the AI prompt quality BEFORE building UI around it. A beautiful Generate button with generic output will undermine the demo.
2. The public page at /p/[username] is your most important page. It is what you show at London Tech Week. Spend disproportionate time on it.
3. Keep Supabase queries in page files for now. When Cursor asks 'show me the dashboard query,' it should find it in one file.
4. Name things for what they do, not what they are. generateProfileContent() beats AIService.generate().
5. The demo is the product. Time the full flow today: signup → fill profile → generate → share link. If it takes more than 5 minutes, you have a UX problem, not a features problem.
6. Technical debt in error handling and logging is fine. Technical debt in security (service role key, RLS) is not fine.

The one thing to internalise: You don't need a perfect product. You need a demonstrable thesis. PersonaPage must answer one question in 30 seconds: "You send different links to different people, and each link shows a different version of you." If anyone understands that immediately — you have a product. Everything else is polish.